↑ Problem -1

① Payment system:-

Payment method (UPI, CARD, PayPal)

⇒ By which any one of this we can finish the process.

Structure

Strategy → payment strategy

Concrete → UPI, CARD, PayPal

Context → Payment processor

Strategy Interface:-

```
class paymentstrategy(ABC):
    def pay(self, amount):
        pass
```

Concrete strategies

```
class UPIpayment(Paymentstrategy):
    def pay(self, amount):
        Print("Processing UPI payment of ₹{amount}")
        Print("validating UPI ID")
        print("payment sucessfully via UPI")

class CardPayment(Payment strategy):
    def pay(self, amount):
        Print("processing card payment of {amount}")
        print("checking card details");
        print("Payment sucessful via card");
```

Scanned with OKEN Scanner

```
class PayPal(PaymentStrategy):
    def pay(self, amount):
        print(f"Processing PayPal payment of {amount}")
        print("Redirecting to PayPal...")
        print("Payment successful via PayPal")
```

Context ..?

```
class PaymentProcessor:
```

It is the main system that uses a strategy, but does not implement the logic itself

```
    def __init__(self, strategy: PaymentStrategy):
        self.strategy = strategy

    def set_strategy(self, strategy: PaymentStrategy):
        self.strategy = strategy

    def process_payment(self, amount):
        self.strategy.pay(amount)
```

usage:

```
Processor = PaymentProcessor(UPIPayment())
Processor.process_payment(500)
```

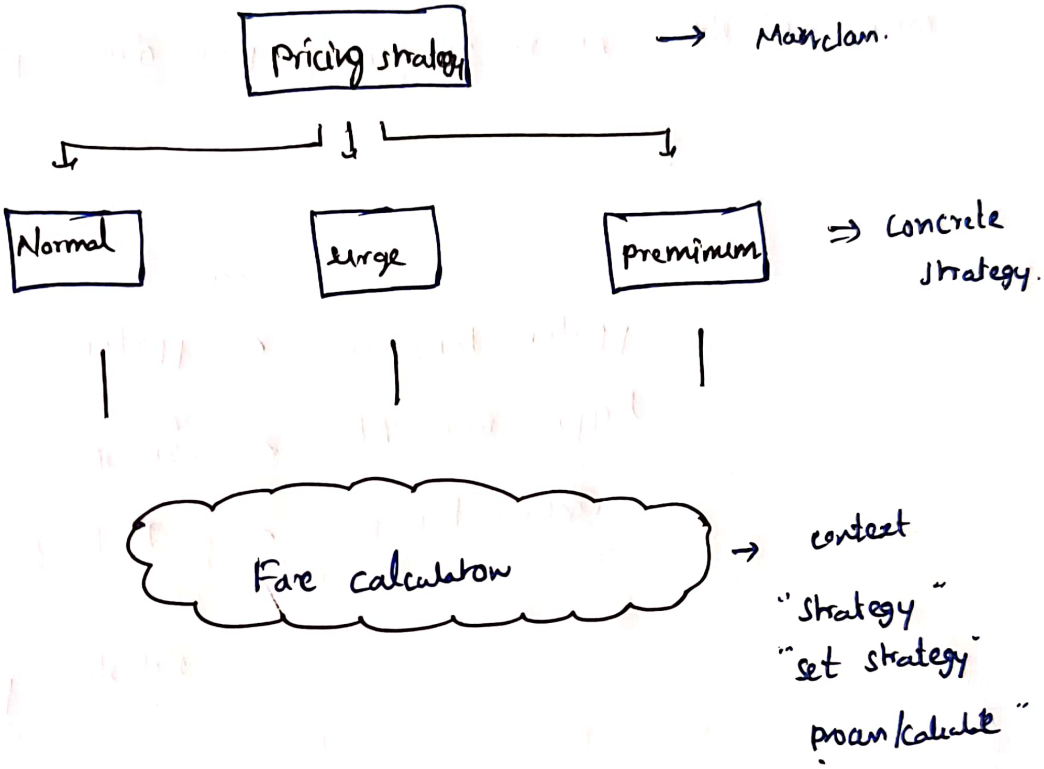
To interchange we will set process...

```
Processor.set_strategy(CardPayment())
Processor.process_payment(1000)
Processor.set_strategy(PayPal())
Processor.process_payment(2000)
```

Scanned with OKEN Scanner

Problem

(2) UBER RIDE pricing system



Scanned with OKEN Scanner